软件设计师（中级）上午场基础知识纯概念速查手册

全面覆盖：数据结构 | 系统结构 | 操作系统 | 数据库 | 网络与安全 | 软工与UML | 设计模式 | 知识产权

一、 数据结构与算法基础

1. 线性表、栈与队列

- **数组与广义表**：数组是静态随机存取结构；稀疏矩阵常采用**三元组表**或**十字链表**进行压缩存储。
- **栈 (Stack)**：先进后出 (LIFO)。常用于表达式求值、函数递归调用、括号匹配、深度优先搜索。
- **队列 (Queue)**：先进先出 (FIFO)。循环队列中通常少用一个位置用于区分“队空”(`front == rear`) 和“队满” (`(rear+1)%size == front`)。
- **串 (String)**：KMP 模式匹配算法的核心是利用 **next 数组**（最大相同前后缀长度）避免主串指针回溯，时间复杂度降低至 O(m+n)。

2. 树与二叉树

- **二叉树核心性质**：第 i 层最多有 2^{i-1} 个节点；深度为 k 的二叉树最多有 $2^k - 1$ 个节点；对任意二叉树，叶子节点数 n_0 = 度为2节点数 n_2 + 1。
- **平衡二叉树 (AVL)**：任意节点的左右子树高度差的绝对值不超过 1。
- **哈夫曼树 (Huffman)**：带权路径长度 (WPL) 最小的树。没有度为 1 的节点；构造时每次选取权值最小的两个节点合并。
- **遍历序列**：**先序**(根-左-右)、**中序**(左-根-右)、**后序**(左-右-根)。中序遍历二叉排序树可得到有序递增序列。

3. 图论

- **存储结构**：邻接矩阵（适合稠密图，空间 $O(n^2)$）、邻接表（适合稀疏图，空间 O(n+e)）。
- **图的遍历**：深度优先搜索（DFS，利用栈）与广度优先搜索（BFS，利用队列）。
- **最小生成树 (MST)**：**Prim 算法**（适合稠密图，加点法）；**Kruskal 算法**（适合稀疏图，加边法）。
- **最短路径**：Dijkstra 算法用于单源最短路径（不支持负权边）；Floyd 算法用于所有节点对间最短路径。

4. 排序与查找

- **查找技术**：折半查找（二分查找）要求线性表必须**顺序存储且有序**，平均查找长度 ASL = $\log_2(n+1) - 1$。
- **哈希表冲突解决**：开放定址法（线性探测、二次探测）、链地址法（拉链法）。
- **排序算法对比**：
  - **稳定排序**：直接插入、冒泡、归并、基数排序。
  - **不稳定排序**：简单选择、希尔、快速、堆排序。
  - **时间复杂度 O(n log n)**：快速排序（最坏 $O(n^2)$）、归并排序、堆排序。

二、 计算机体系结构与硬件

1. 体系结构分类与 Flynn 模型

- **Flynn 分类**：SISD（单指令单数据，单核CPU）、SIMD（单指令多数据，向量/阵列处理机）、MISD（仅理论）、MIMD（多指令多数据，多核/多处理器/分布式）。
- **CISC 与 RISC**：
  - **CISC (复杂指令集)**：指令多且变长、寻址多、使用微程序控制、研制周期长。
  - **RISC (精简指令集)**：指令少且定长、寻址少、硬布线逻辑控制、大量使用通用寄存器、极易实现流水线。

2. 存储层次与总线

- **Cache (高速缓存)**：采用 **SRAM** 制造，解决 CPU 与主存速度不匹配问题；对程序员完全透明。
- **Cache 映射方式**：直接映射（冲突率高）、全相联映射（冲突率低但比较器复杂）、组相联映射（折中方案）。
- **总线**：数据总线（双向，决定字长）、地址总线（单向，决定寻址空间 2^N B）、控制总线（传输控制与中断信号）。

三、 操作系统知识

1. 进程管理与同步

- **进程与线程**：进程是**资源分配与独立拥有**的基本单位；线程是**独立调度与执行**的基本单位，共享进程的资源。
- **进程状态变迁**：就绪 → 运行（获得CPU）；运行 → 就绪（时间片用完）；运行 → 阻塞（等待资源/IO）；阻塞 → 就绪（资源到位/IO完成）。
- **PV 操作**：P(S) 操作将信号量 S-1，若 S < 0 则进程阻塞；V(S) 操作将信号量 S+1，若 S ≤ 0 则唤醒阻塞队列中的一个进程。
- **死锁四大必要条件**：互斥条件、请求与保持条件、不剥夺条件、环路等待条件。
- **死锁避免策略**：动态预防策略，最典型的是**银行家算法**（确保系统处于安全状态，不发生死锁）。

2. 存储与文件管理

- **页面置换算法**：
  - **FIFO (先进先出)**：可能引发 **Belady 异常**（即分配的页框数增加，缺页率反而上升）。
  - **LRU (最近最少使用)**：依据过去的使用情况置换近期最少使用的页面。
  - **OPT (最佳置换)**：淘汰未来最长时间不被使用的页，属理想算法，不可实际实现。
- **索引文件结构**：采用多级索引（直接索引、一级间接索引、二级间接索引等），以扩大单个文件可寻址的容量上限。
- **磁盘调度算法**：FCFS（先来先服务）、SSTF（最短寻道时间优先，易导致饥饿）、SCAN（电梯顺向扫描算法）。

四、 数据库系统概念

1. 三级模式与两级映射

- **三级模式结构**：外模式（视图/用户级）、概念模式（基本表/全局逻辑级）、内模式（存储文件/物理级）。
- **两级数据独立性**：
  - **外模式/概念模式映射**：提供**逻辑数据独立性**（当概念模式改变，只需修改映射，外模式不变）。
  - **概念模式/内模式映射**：提供**物理数据独立性**（当物理存储改变，只需修改映射，概念模式不变）。

2. 规范化理论（范式）

- **1NF (第一范式)**：属性不可再分（符合原子性）。
- **2NF (第二范式)**：满足 1NF，且消除非主属性对码的**部分函数依赖**。
- **3NF (第三范式)**：满足 2NF，且消除非主属性对码的**传递函数依赖**。
- **BCNF (巴斯范式)**：消除所有主属性或非主属性对码的部分与传递函数依赖（即所有决定因素都包含码）。

3. 事务 ACIDS 特性与锁

- **ACID 特性**：**原子性** (Atomicity，Undo日志)、**一致性** (Consistency)、**隔离性** (Isolation，锁机制)、**持久性** (Durability，Redo日志)。
- **并发不一致问题**：丢失修改、不可重复读、读“脏”数据。通过封锁协议解决：
  - **一级封锁协议**：防丢失修改（加 X 锁至事务结束）。
  - **二级封锁协议**：防丢失修改、防读脏数据（读完即释放 S 锁）。
  - **三级封锁协议**：防丢失修改、防读脏数据、防不可重复读（加 S 锁至事务结束）。

五、 设计模式（23种模式全梳理）

1. SOLID 设计原则

- **单一职责原则 (SRP)**：一个类应该仅有一个引起它变化的原因。
- **开闭原则 (OCP)**：对扩展开放，对修改关闭（核心总原则）。
- **里氏替换原则 (LSP)**：子类必须能够替换掉它们的基类（程序行为不改变）。
- **依赖倒置原则 (DIP)**：高层模块不应依赖底层模块，二者都应依赖其抽象；抽象不依赖细节。
- **接口隔离原则 (ISP)**：类间的依赖关系应建立在最小的接口上，不应强迫依赖不需要的接口。
- **迪米特法则 (LoD)**：最少知识原则，一个对象应对其他对象保持最少的了解。

2. 23种设计模式速查表

分类	模式名称	核心作用与应用场景纯概念
创建型 (对象的构建)	单例模式 (Singleton)	保证一个类仅有一个实例，并提供一个全局访问点。
	工厂方法 (Factory Method)	定义创建对象的接口，由子类决定具体实例化哪一个类。
	抽象工厂 (Abstract Factory)	提供一个接口，用于创建一系列相关或相互依赖的对象家族。
	建造者模式 (Builder)	将复杂对象的构建与它的表示分离，使同样的构建过程可创建不同的表示。
	原型模式 (Prototype)	通过复制现有实例（克隆）来创建新的对象，避免重新初始化。
结构型 (类与对象的组合)	适配器模式 (Adapter)	将一个类的接口转换成客户希望的另一个接口，解决接口不兼容问题。
	桥接模式 (Bridge)	将抽象部分与它的实现部分分离，使它们都可以独立地变化。
	组合模式 (Composite)	将对象组合成树形结构以表示“部分-整体”层次，统一对待单个与组合对象。
	装饰器模式 (Decorator)	动态地给一个对象添加额外的职责（比生成子类继承更灵活）。
	外观模式 (Facade)	为子系统中的一组接口提供一个一致的高层界面，降低子系统耦合度。
	享元模式 (Flyweight)	运用共享技术有效地支持大量细粒度对象的复用，节省内存开销。
	代理模式 (Proxy)	为其他对象提供一种代理以控制对这个对象的访问（如远程代理、延迟加载）。
行为型 (算法与职责分配)	策略模式 (Strategy)	定义一系列算法，把它们一个个封装起来，并使它们可以相互替换。
	观察者模式 (Observer)	定义一对多的依赖关系，当一个对象改变状态，所有依赖者都会自动收到通知并更新。
	状态模式 (State)	允许一个对象在其内部状态改变时改变它的行为，看起来就像改变了类。
	模板方法 (Template Method)	定义一个操作中的算法骨架，而将一些步骤延迟到子类中实现。

六、 计算机网络与信息安全

1. 网络体系结构与典型协议

- **应用层协议**：HTTP (80), HTTPS (443, SSL/TLS), FTP (20数据/21控制), DNS (53, UDP/TCP), SMTP (25, 发邮件), POP3 (110, 接收邮件), DHCP (67/68, 动态分配IP)。
- **传输层协议**：
  - **TCP**：面向连接、可靠交付（超时重传、滑动窗口流量控制、拥塞控制）、全双工、字节流。
  - **UDP**：无连接、不可靠、尽最大努力交付、面向报文、开销小速度快。
- **网络层协议**：IP、ICMP (Ping与Tracert工具依据)、ARP (根据 IP 地址查找 MAC 地址)、RARP。

2. 加密技术与数字签名

- **对称加密**：加解密使用同一密钥。速度快，适合加密大文件。典型算法：AES、DES、3DES、RC4。
- **非对称加密**：使用公钥加密、私钥解密（或反之）。速度慢。典型算法：RSA、ECC。
- **数字签名**：**发送方使用自己的私钥**对消息摘要加密。作用：保证真实性、完整性、不可否认性（防抵赖）。
- **数字证书**：由权威机构 (CA) 签发，用于确认公钥所有者的合法真实身份。

七、 软件工程与 UML 建模

1. 开发过程模型与敏捷

- **瀑布模型**：阶段明确、线性顺序；适合需求明确固定的项目。
- **螺旋模型**：综合了瀑布与原型模型，**最大特色是引入了风险分析**，适合大型复杂系统。
- **敏捷开发**：强调适应性而非预设性。包含 **Scrum**（Sprint迭代、每日站会）与 **XP (极限编程)**（结对编程、测试驱动开发 TDD、重构、持续集成）。
- **CMMI 能力成熟度**：初始级 → 已管理级（项目级） → 已定义级（组织级） → 定量管理级 → 优化级。

2. UML 视图与图分类

- **静态图（结构图）**：类图（Class，描述类及静态关系）、对象图、包图、组件图、部署图（描述物理硬件与节点分配）。
- **动态图（行为图）**：
  - **用例图 (Use Case)**：展现参与者 (Actor) 与用例间关系。包含关系：包含 (<<include>>)、扩展 (<<extend>>)、泛化。
  - **顺序图 / 时序图 (Sequence)**：强调消息传递的**时间顺序**。
  - **通信图 / 协作图 (Communication)**：与顺序图同构，强调对象的**空间组织结构**。
  - **状态图 (State)**：展示单个对象在生命周期中因事件触发的状态变迁。
  - **活动图 (Activity)**：展示业务流程或算法步骤（支持并发分流与汇合，类似流程图）。

八、 知识产权与标准化

- **软件著作权归属**：
  - **职务作品**：主要利用单位物质技术条件并由单位承担责任的，著作权归**单位**所有。
  - **委托开发**：合同有约定的按约定；合同未约定或约定不明确的，著作权归**受托方（开发者）**所有。
- **保护期限与产生时间**：软件著作权自**软件开发完成之日起自动产生**；自然人终身及死后 50 年，法人/组织发表后 50 年。
- **工业产权**：发明专利保护期 **20年**（自申请日起算）；商标权保护期 **10年**（按申请在先原则，可无限续展）。